

위성유도제어 – LAB#5

2022311224 이진진

1) Explain the following code; animation_3axis_sat.m.

animation_3axis_sat.m 코드는 위성의 자세 데이터를 3차원 그래프에 시각적으로 표현해주는 함수이다. torque 발생과 비행체의 자세 조정으로 인한 축의 변화를 한눈에 볼 수 있다.

```

1  % 위성 각 축의 시간에 따른 자세를 3차원으로 animaiton을 보기위한 M-file
2  % file name : animation_3axis_sat.m
3
4  %-----
5  % Make satellite shape (3-axis satellite)  위성의 본체 및 태양전지판을 모델링하는 함수를 설정한다
6  h_fig2=figure(2);
7  set(h_fig2,'numbertitle','off','name','By Park In-Kwan')
8  set(h_fig2,'pos',[166 146 467 377],'color','w')  figure2의 설정 정보 및 작성할 문구의 내용을 설정한다
9
10 a=[0 0.75 1.5 1.5 0.75 0 -0.75 -1.5 -1.5 -0.75];
11 b=[1.5 1.5 1.5 -1.5 -1.5 -1.5 -1.5 -1.5 1.5 1.5];
12 A=[a b];
13 c1=zeros(10,1)+1.5;
14 c2=zeros(10,1)-1.5;
15 Ah=[a' b' c1];
16 Al=[a' b' c2];
17 iniver=[Ah; Al];
18 face=[10 9 8 7;1 10 7 6;2 1 6 5;3 2 5 4;20 19 18 17;11 20 17 16;12 11 16 15;
19       13 12 15 14;1 2 12 11;2 3 13 12;3 4 14 13;4 5 15 14;5 6 16 15;6 7 17 16;7 8 18 17;8 9 19 18;9 10 20 19;10 1 11 20];
20 patch_color=0.9*ones(10,3);
21 patch_hnd=patch('vertices',iniver,'faces',face,'facecolor','b','edgecolor','y','erasemode','background');
22
23 %right pannel design
24 arp=[0 1.5 1.5 0 -1.5 -1.5]
25 brp=[3 3 8 8 8 3]
26 Arp=[arp brp];
27 crp1=zeros(6,1)+0.1;
28 crp2=zeros(6,1)-0.1;
29 Arph=[arp' brp' crp1];
30 Arpl=[arp' brp' crp2];
31 panr=[Arph; Arpl];
32 facepr=[6 5 4 1;1 4 3 2;12 11 10 7;7 10 9 8;1 2 8 7;2 3 9 8;3 4 10 9; 4 5 11 10;5 6 12 11;6 1 12 7];
33 patchpr=patch('vertices',panr,'faces',facepr,'facecolor','b','edgecolor','y','erasemode','background');
34
35 %left pannel design
36 alp=[0 1.5 1.5 0 -1.5 -1.5]
37 blp=[-3 -3 -8 -8 -8 -3]
38 Alp=[alp blp];
39 clp1=zeros(6,1)+0.1;
40 clp2=zeros(6,1)-0.1;
41 Alph=[alp' blp' clp1];
42 Alpl=[alp' blp' clp2];
43 panl=[Alph; Alpl];
44 facepl=[6 5 4 1;1 4 3 2;12 11 10 7;7 10 9 8;1 2 8 7;2 3 9 8;3 4 10 9; 4 5 11 10;5 6 12 11;6 1 12 7];
45 patchpl=patch('vertices',panl,'faces',facepl,'facecolor','b','edgecolor','y','erasemode','background');
46
47 %right bar
48 arb=[0.1 0.1 -0.1 -0.1];
49 brb=[1.5 3 3 1.5];
50 crb1=zeros(4,1)+0.1;
51 crb2=zeros(4,1)-0.1;
52 Arbh=[arb' brb' crb1];
53 Arbl=[arb' brb' crb2];
54 barr=[Arbh; Arbl];
55 facebr=[1 2 3 4;5 6 7 8;1 2 6 5;2 3 7 6;3 4 8 7;4 1 5 8];
56 patchbr=patch('vertices',barr,'faces',facebr,'facecolor','b','edgecolor','k','erasemode','background');
57
58 %left bar
59 alb=[0.1 0.1 -0.1 -0.1];
60 blb=[-1.5 -3 -3 -1.5];
61 clb1=zeros(4,1)+0.1;
62 clb2=zeros(4,1)-0.1;
63 Albh=[alb' blb' clb1];
64 Albl=[alb' blb' clb2];
65 barl=[Albh; Albl];
66 facebl=[1 2 3 4;5 6 7 8;1 2 6 5;2 3 7 6;3 4 8 7;4 1 5 8];
67 patchbl=patch('vertices',barl,'faces',facebl,'facecolor','b','edgecolor','k','erasemode','background');
68 view(3)
69 axis equal
70 set(gca,'xlim',[-10 10],'ylim',[-10 10],'zlim',[-10 10])

```

위성체의 본체, 오른쪽 패널, 왼쪽 패널 세 부분으로 나누어 변수를 입력하였으며 위성체의 가로, 세로, 높이 등 물리적 크기와 위성체의 본체 및 윤곽의 색상 등 시각적 정보로 구분되어 있다. 각각의 함수에 대한 자세한 설명은 위의 첨부사진에 작성하였다.

```

편집기 - C:\Users\이진진\Documents\
animation_3axis_sat.m * x +
75 N=length(out.Euler_angles);
76 for index=1:N
77     phi=out.Euler_angles(index,1)*pi/180;
78     theta=out.Euler_angles(index,2)*pi/180;
79     psi=out.Euler_angles(index,3)*pi/180;
80
81     % rotation matrix ( Orbit frame component ---> Body frame component)
82     R=rotmat(phi,theta,psi);
83     % R' : Body frame component ---> Orbit frame component, (an inertial component, if Wo = 0)
84     Ver=R'*iniver';
85     Ver=Ver';
86     set(patch_hnd,'vertices',Ver)
87
88     Verpr=R'*panr';
89     Verpr=Verpr';
90     set(patchpr,'vertices',Verpr)
91
92     Verpl=R'*panl';
93     Verpl=Verpl';
94     set(patchpl,'vertices',Verpl)
95
96     Verbr=R'*barr';
97     Verbr=Verbr';
98     set(patchbr,'vertices',Verbr)
99
100     Verbl=R'*barl';
101     Verbl=Verbl';
102     set(patchbl,'vertices',Verbl)
103
104
105     drawnow;
106     pause(0.01)
107 end
108 %end of code -----
109

```

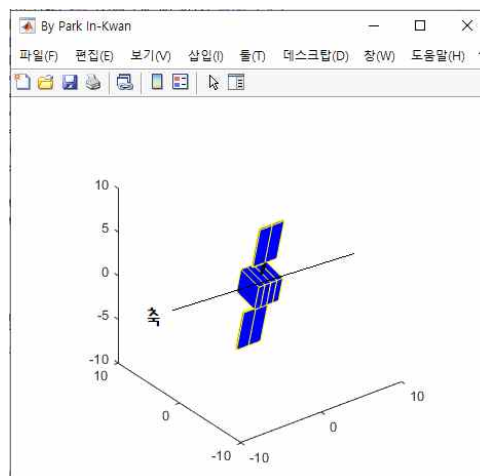
시뮬레이션 결과에 따른 3차원 그래프 상의 위치 변화를 설정한다

phi, theta, psi 세 변수에 대해 작성하였으며 단위 변환을 위해 pi/180을 곱하였다

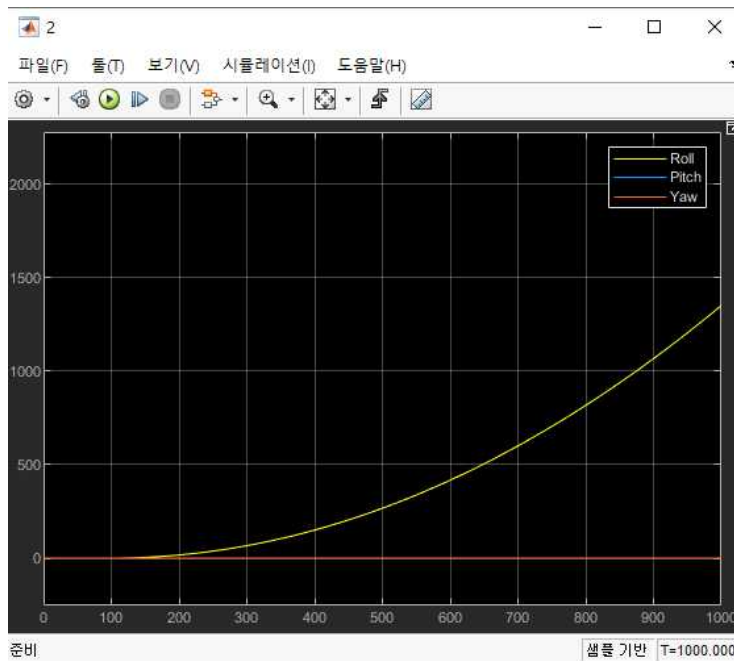
orbit 에서 body frame으로의 변환을 위해 Rotation matrix를 사용

orbit에서 body frame으로의 변환을 위한 R' 함수이며 위에서 다른 위성체의 다섯 영역에 대하여 작성한다

위성체의 물리, 시각적 정보를 입력한 후에는 시간에 따른 frame의 변화에 대한 코드를 작성한다. 이를 위해 rotation matrix를 사용하는데 교안을 참조하여 세 변수(phi, theta, psi)에 대한 함수를 rotmat.m 파일로 별도 작성하였고 기존의 파일명을 새로 작성한 파일명으로 변경하였다. 시뮬레이션 결과로 얻은 Euler angle을 이용하여 시간에 따라 Body frame에서 Orbit frame으로 변환시킨 위성의 형상을 표현한다. simulink와의 호환을 위해 코드의 일부 (Euler_angles -> out.Euler_angles)를 수정하였다. 시뮬레이션 결과 아래 그림과 같이 roll 축 자세 변화를 볼 수 있다.

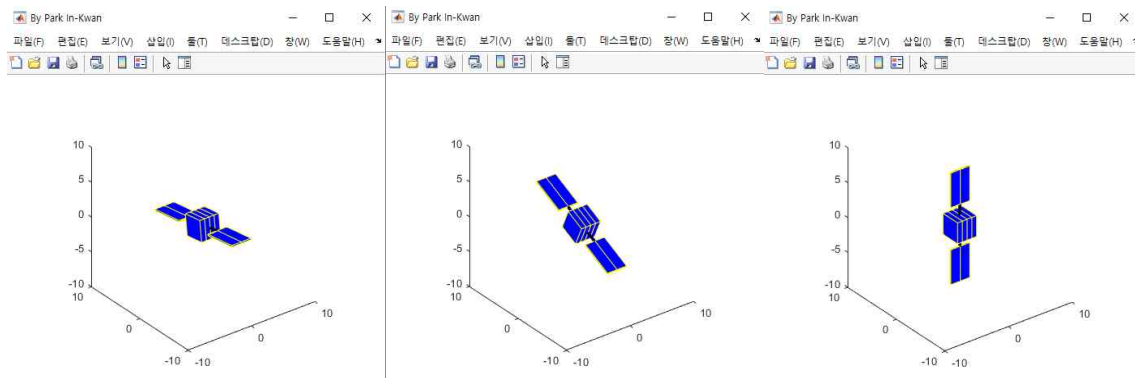


2-a) Simulate the attitude motion using the Simulink code.



100초 후, Roll 축 방향에 step disturbance torque가 1 Nm 작용했을 때의 결과이다.

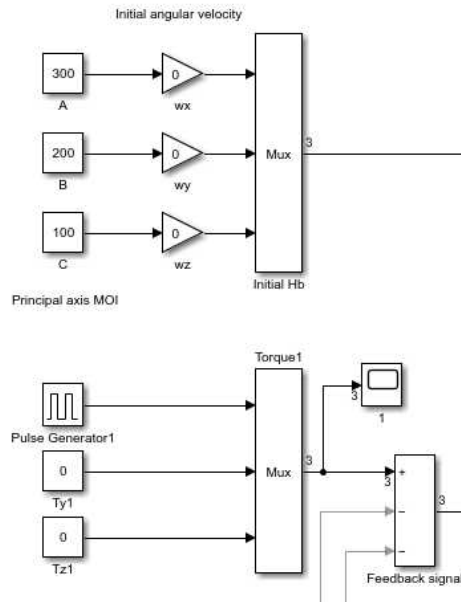
2-b) Show the animation of the motion using animation_3axis_sat.m.



시뮬레이션 결과, 그림과 같이 비행체가 Roll 축에 대해서 시계방향으로 회전하는 모습을 볼 수 있다. 회전을 시작하는 시간은 torque를 가한 시점인 시뮬레이션 시작 후 100초 부터이며 그 전의 움직임은 없는 것으로 확인된다.

2-c) Explain the rotational motion.

문제에서 주어진 초기의 위치와 angular velocity는 모두 (0,0,0) 이므로 torque가 가해지기 전까지의 운동은 없다. 100초부터는 roll축에 대해 torque가 발생하는데 이에 대한 결과를 시뮬레이션 하기 위해 simulink에서 아래와 같이 inertia tensor와 함께 지연 시간 100초를 입력하여 pulse generator를 가동시켰다. 따라서 100초 후부터는 roll 축을 중심으로 회전하며 angular velocity가 일정하게 증가할 것으로 예상해볼 수 있다.



3-a) Estimate the values of necessary parameters for a PD controller.

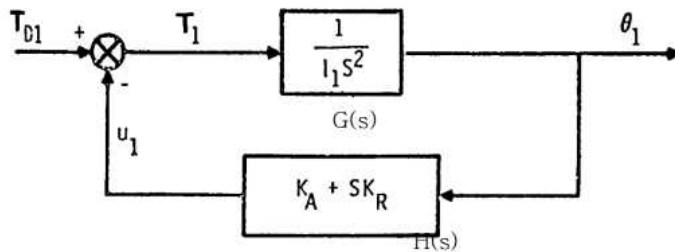


Fig 4.25 Approximation of roll channel ACS

문제에서 주어진 조건에 맞게 PD control을 하기 위해 Ka와 Kr을 구한다. 이 과정을 위한 system servomechanism에 대해서는 강의 자료의 그림 4.25를 통해 진행했다. 이 시스템에서 TF(Transfer function)는 output/input 으로 계산된 $1 / (I s^2 + K_R s + K_A)$ 이며 이 때, critical damping으로 control한다고 가정하여 Ka와 Kr, 그리고 $\theta(s)/T_d(s)$ 를 구하면 다음과 같다.

$$\sqrt{K_R^2 - 4IK_A} = 0 \Rightarrow K_A = \frac{K_R^2}{4I}$$

$$\Rightarrow s_{1,2} = \frac{-K_R}{2I} \equiv -\frac{1}{\tau} \quad ; \text{time const.}$$

$$K_R = \frac{2I}{\tau}$$

Thus,

$$\begin{aligned} \frac{\theta(s)}{T_D(s)} &= \frac{1}{Is^2 + K_R s + K_A} = \frac{1}{Is^2 + \frac{2I}{\tau}s + \frac{1}{4I} \frac{4I^2}{\tau^2}} \\ &= \frac{1}{I\left(s^2 + \frac{2}{\tau}s + \frac{1}{\tau^2}\right)} = \frac{1}{I\left(s + \frac{1}{\tau}\right)^2} = \frac{1}{I \frac{1}{\tau^2} (\tau s + 1)^2} \end{aligned}$$

이제, 출력 값을 Laplace 변환으로 시간 t에 대해 표현할 수 있다.

$$\theta(t) = \frac{T_D}{I} \left[\tau^2 \left(1 - e^{-\frac{t}{\tau}} \right) - \tau t e^{-\frac{t}{\tau}} \right]$$

이 값의 최대 값을 구하면 아래의 식과 같다.

$$\theta(t)_{\text{maximum}} \rightarrow \theta(t)_{t=\infty} \rightarrow \frac{\tau^2 T_D}{I} \quad ; \text{steady-state value (maximum value)}$$

$$\theta(\infty) = \lim_{s \rightarrow 0} s \theta(s) = \lim_{s \rightarrow 0} \frac{s \tau^2 T_D}{Is(\tau s + 1)^2} = \frac{\tau^2 T_D}{I} \quad ; \text{final value theorem}$$

주어진 초기 값(I=300, Td=1)으로 계산을 하면 다음과 같은 결과를 얻을 수 있다.

$$\theta_{\text{max}} = \frac{\pi}{1800} \quad K_A = \frac{1800}{\pi} \quad K_R = 600 \sqrt{\frac{6}{\pi}}$$

3-b) Figure out analytically the history of control.

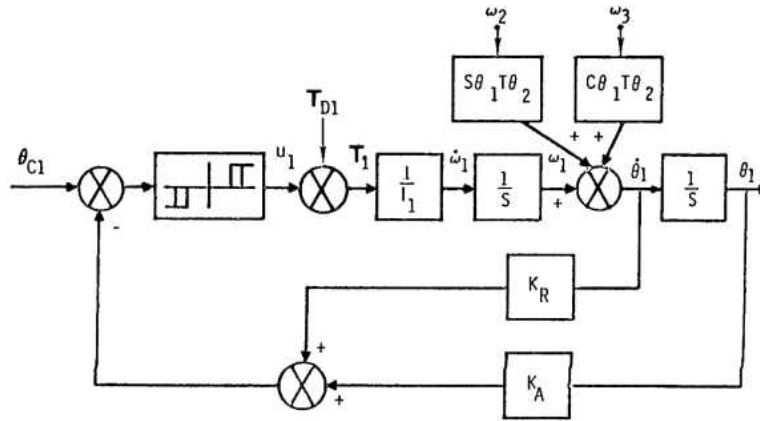


Fig 4.24 Simplified roll channel reaction attitude control system(ACS)

roll 값을 조정하기 위한 servomechanism으로 강의 자료의 그림 4.24를 사용한다.

$$u(s) = (K_A + sK_R)\theta(s)$$

$$u(s) = \left(\frac{I}{\tau^2} + s \frac{2I}{\tau} \right) \frac{\tau^2 T_D}{Is(\tau s + 1)^2}$$

$$= \frac{(1 + 2\tau s)T_D}{s(\tau s + 1)^2}$$

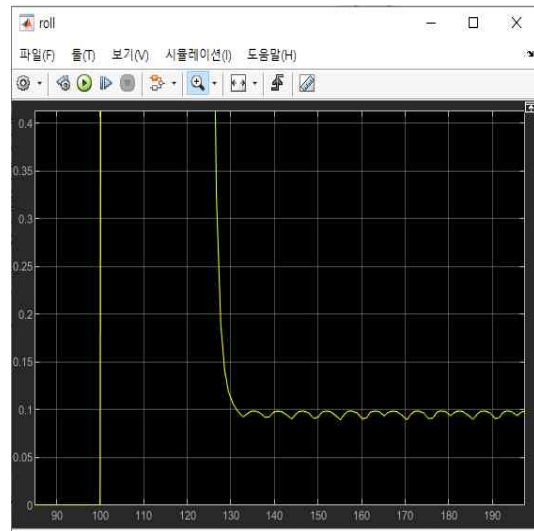
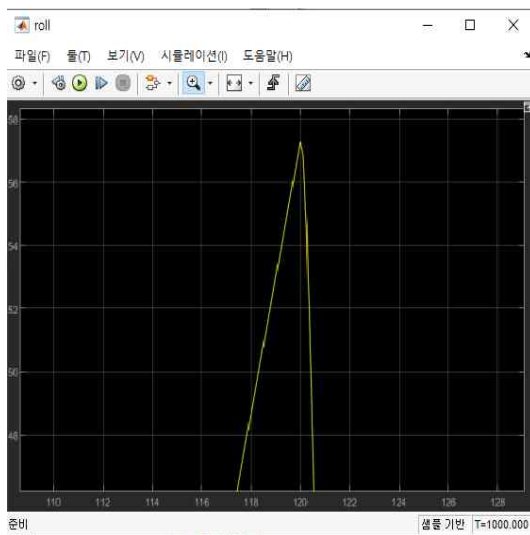
$$\lim_{t \rightarrow \infty} u(t) = \lim_{s \rightarrow 0} su(s) = T_D$$

$$u(t) = T_D \left[\left(1 - e^{-\frac{t}{\tau}} \right) - \frac{t}{\tau} e^{-\frac{t}{\tau}} \right]$$

문제 b와 마찬가지로 Laplace 변환을 사용해 $u(t)$ 를 구하면 위와 같은 식을 얻을 수 있다. 이 식에서 t 가 최대일 때, 최종 값은 disturbance torque 만큼이 feedback signal로 보존되어 입력되는 것을 확인할 수 있다.

$$U_{roll}(t)_{t \rightarrow \infty} = T_D$$

3-c) Perform simulations and explain the results in detail.



시뮬레이션 결과, 실행 후 100초 후에 Disturbance torque가 작용하였고 120초 후에 0.1 degree 이내의 error로 줄인 모습을 볼 수 있다. 또한 0.1 부분을 확대하였더니 해당 부근에서 각도가 유지되는 것을 확인할 수 있었다.

3-d) Explain the parameters and results according as maximum roll error varies from 0.001, 0.01, 0.1, 1 degree.

maximum roll error (degree)	K_r	K_a
0.001	$600 \sqrt{6/\pi}$	$180000/\pi$
0.01	$1200 \sqrt{15/\pi}$	$18000/\pi$
0.1	$600 \sqrt{6/\pi}$	$1800/\pi$
1	$120 \sqrt{15/\pi}$	$180/\pi$

문제 3-a)에서 계산한 식을 이용하여 roll error의 변화에 따른 K_r 과 K_a 를 계산해보면 위와 같은 결과가 나온다.